Résolution du CVRP par Colonies de Fourmis améliorée par Q-Learning

Projet d'Optimisation Combinatoire
Apport du Machine Learning pour l'amélioration des métaheuristiques

Grégory MENEUS Arnaud GRIMBERT Odin LANDU

Année universitaire 2025–2026

Résumé

Dans ce rapport, on présente notre travail sur la résolution du problème de tournées de véhicules avec contraintes de capacité (CVRP). On a choisi d'implémenter une approche par colonies de fourmis (ACO), puis de l'améliorer en utilisant du Q-Learning pour ajuster automatiquement les paramètres de l'algorithme pendant la recherche. L'idée de base, c'est que plutôt que de fixer les paramètres de l'ACO une fois pour toutes, on laisse un agent d'apprentissage par renforcement apprendre quelle stratégie adopter selon l'état de la recherche. On compare les deux approches sur plusieurs instances de tailles différentes, et on discute de ce que le machine learning apporte vraiment dans ce contexte.

Table des matières

1 Introduction **2**

2 Modélisation du problème **2**
2.1 Description du CVRP . . . 2
2.2 Formulation mathématique . . . 2
2.3 Difficulté et intérêt . . . 3

3 Métaheuristique : Colonies de fourmis (ACO) **3**
3.1 Principe général . . . 3
3.2 Construction d'une solution . . . 3
3.3 Mise à jour des phéromones . . . 4
3.4 Paramètres utilisés . . . 4
3.5 Pseudocode . . . 5

4 Intégration du Q-Learning **5**
4.1 Pourquoi le Q-Learning ? . . . 5
4.2 Formulation état–action–récompense . . . 5
4.2.1 Espace d'états (27 états) . . . 5
4.2.2 Espace d'actions (6 stratégies) . . . 6
4.2.3 Fonction de récompense . . . 6
4.2.4 Mise à jour Q-Learning . . . 6
4.3 Phase de warm-up . . . 7

4.4 Recherche locale 2-opt 7

5 Implémentation **7**
5.1 Architecture du code 7
5.2 Reproductibilité 7

6 Expérimentations **8**
6.1 Protocole expérimental 8
6.2 Résultats 8
6.3 Analyse des résultats 8
6.3.1 Qualité des solutions 8
6.3.2 Stabilité 8
6.3.3 Temps de calcul 9
6.4 Visualisation de la convergence 9

7 Discussion **9**
7.1 Apports du Q-Learning 9
7.2 Limites 10
7.3 Pistes d'amélioration 10

8 Conclusion **11**

1 Introduction

Les problèmes d'optimisation combinatoire sont partout dans le monde réel : logistique, planification, allocation de ressources, etc. Parmi eux, le problème de tournées de véhicules (VRP) est un classique, et sa variante avec capacités (CVRP) est celle qu'on rencontre le plus souvent en pratique.

Le CVRP consiste à trouver les meilleurs itinéraires pour une flotte de véhicules qui doivent livrer des clients à partir d'un dépôt central. Chaque véhicule a une capacité maximale qu'il ne faut pas dépasser. C'est un problème NP-difficile, ce qui veut dire qu'il n'existe pas d'algorithme capable de trouver la solution optimale en temps raisonnable dès que le nombre de clients augmente.

Pour résoudre ce type de problème, on utilise souvent des métaheuristiques. Ces algorithmes ne garantissent pas de trouver l'optimum, mais ils donnent de bonnes solutions en un temps acceptable. Le souci, c'est que leurs performances dépendent beaucoup de leurs paramètres, et les régler correctement est souvent un travail fastidieux qui se fait par essais-erreurs.

C'est là que le machine learning entre en jeu. L'idée qu'on explore dans ce projet, c'est d'utiliser un algorithme d'apprentissage par renforcement (le Q-Learning) pour que l'algorithme apprenne lui-même à ajuster ses paramètres au fil de la résolution. Au lieu de chercher manuellement les "bons" paramètres, on laisse l'algorithme s'adapter dynamiquement.

Dans ce rapport, on commence par présenter le CVRP et sa modélisation. Ensuite, on décrit notre implémentation de l'ACO classique, puis l'intégration du Q-Learning. Enfin, on présente nos résultats expérimentaux et on discute de ce qui marche et de ce qui marche moins bien.

2 Modélisation du problème

2.1 Description du CVRP

Le CVRP (Capacitated Vehicle Routing Problem) se définit comme suit :

- On a un **dépôt central** d'où partent et reviennent tous les véhicules.
- On a n **clients**, chacun situé à une position (x_i, y_i) et ayant une demande d_i à satisfaire.
- On dispose d'une flotte de véhicules identiques, chacun avec une **capacité maximale** Q.
- Chaque client doit être visité **exactement une fois** par un seul véhicule.
- La charge totale d'un véhicule sur sa tournée ne doit pas dépasser Q.

2.2 Formulation mathématique

On modélise le problème sur un graphe complet $G = (V, E)$ où $V = \{0, 1, \ldots, n\}$ (0 étant le dépôt) et E l'ensemble des arêtes. La distance entre deux nœuds i et j est la distance euclidienne :

$$d_{ij} = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2} \tag{1}$$

Une solution est un ensemble de routes $R = \{r_1, r_2, \ldots, r_K\}$ où chaque route r_k est

une séquence de clients qui commence et finit au dépôt. La fonction objectif à minimiser est la distance totale :

$$\min \sum_{k=1}^{K} \left(d_{0,r_k(1)} + \sum_{i=1}^{|r_k|-1} d_{r_k(i),r_k(i+1)} + d_{r_k(|r_k|),0} \right) \tag{2}$$

sous les contraintes :

$$\sum_{i \in r_k} d_i \leq Q \quad \forall k \in \{1, \ldots, K\} \tag{3}$$

$$\bigcup_{k=1}^{K} r_k = \{1, \ldots, n\} \tag{4}$$

$$r_k \cap r_l = \emptyset \quad \forall k \neq l \tag{5}$$

2.3 Difficulté et intérêt

Le CVRP est NP-difficile. Concrètement, pour n clients, le nombre de solutions possibles explose de façon factorielle. Même avec 50 clients, on ne peut pas énumérer toutes les solutions. C'est ce qui rend les métaheuristiques intéressantes : elles permettent d'explorer intelligemment cet espace gigantesque.

Le CVRP a aussi un intérêt pratique évident. C'est exactement le problème que rencontrent les entreprises de livraison : comment organiser les tournées pour minimiser les coûts tout en respectant la capacité des camions ? Des entreprises comme Amazon ou UPS investissent des millions dans la résolution de ce type de problème.

3 Métaheuristique : Colonies de fourmis (ACO)

3.1 Principe général

L'optimisation par colonies de fourmis (ACO) s'inspire du comportement des fourmis dans la nature. Quand des fourmis cherchent de la nourriture, elles déposent des phéromones sur leur chemin. Les chemins les plus courts accumulent plus de phéromones (parce que les fourmis y passent plus souvent), ce qui attire davantage de fourmis : c'est un mécanisme d'auto-renforcement.

Dans notre cas, les "fourmis" sont des agents qui construisent des solutions au CVRP. Elles choisissent le prochain client à visiter en se basant sur deux critères :

— Les **phéromones** τ_{ij} : plus il y a de phéromone sur un arc, plus il est attractif (c'est la mémoire collective).

— L'**heuristique** $\eta_{ij} = 1/d_{ij}$: les clients proches sont naturellement préférés.

3.2 Construction d'une solution

Chaque fourmi construit une solution complète. Elle part du dépôt et, tant qu'il reste des clients non visités, elle choisit le prochain client selon la probabilité :

$$p_{ij} = \frac{[\tau_{ij}]^\alpha \cdot [\eta_{ij}]^\beta}{\sum_{l \in \mathcal{N}_i} [\tau_{il}]^\alpha \cdot [\eta_{il}]^\beta} \tag{6}$$

où $\mathcal{N}_i$ est l'ensemble des clients encore accessibles (non visités et dont la demande ne fait pas dépasser la capacité), α contrôle l'importance des phéromones et β l'importance de l'heuristique.

Quand une fourmi ne peut plus ajouter de client à sa route courante (à cause de la capacité), elle retourne au dépôt et commence une nouvelle route.

3.3 Mise à jour des phéromones

Après que toutes les fourmis aient construit leur solution, on met à jour les phéromones en deux étapes :

Évaporation : les phéromones diminuent sur tous les arcs pour éviter de rester bloqué sur de mauvaises solutions :

$$\tau_{ij} \leftarrow (1 - \rho) \cdot \tau_{ij} \tag{7}$$

Dépôt (stratégie élitiste) : seules les meilleures fourmis (le tiers supérieur) déposent des phéromones, et la meilleure solution globale reçoit un bonus supplémentaire :

$$\tau_{ij} \leftarrow \tau_{ij} + \sum_{k \in \text{élite}} w_k \cdot \frac{Q_{\text{phéro}}}{C_k} \tag{8}$$

où C_k est le coût de la solution de la fourmi k et w_k un poids décroissant selon le rang.

3.4 Paramètres utilisés

Le tableau 1 résume les paramètres de notre ACO.

TABLE 1 – Paramètres de l'ACO classique

Paramètre	**Valeur**	**Rôle**
α	1.0	Poids des phéromones
β	3.0	Poids de l'heuristique
ρ	0.1	Taux d'évaporation
$Q_{\text{phéro}}$	100	Facteur de dépôt
$\tau_{\min}$	0.1	Phéromone minimale
$\tau_{\max}$	10.0	Phéromone maximale
Nombre de fourmis	20	Par itération
Itérations max	150	Critère d'arrêt

On a choisi $\beta > \alpha$ pour que l'heuristique de distance ait plus d'influence que les phéromones au début, quand les phéromones ne contiennent pas encore beaucoup d'information. Le taux d'évaporation de 0.1 est un compromis classique entre exploration et exploitation.

3.5 Pseudocode

Algorithm 1 ACO classique pour le CVRP

Entrées : Instance CVRP, paramètres α, β, ρ, Q
Sorties : Meilleure solution trouvée

```
Initialiser la matrice de phéromones τ_ij = 1.0
pour t = 1 à T_max faire
    pour chaque fourmi k faire
        Construire une solution S_k (choix probabiliste + contrainte de capacité)
    fin pour
    Identifier les meilleures solutions (élite)
    Évaporer : τ_ij ← (1 − ρ) · τ_ij
    Déposer : phéromones par les fourmis élites + bonus meilleur global
    Mettre à jour la meilleure solution globale
fin pour
retourner meilleure solution
```

4 Intégration du Q-Learning

4.1 Pourquoi le Q-Learning ?

On a choisi le Q-Learning pour plusieurs raisons. D'abord, c'est un algorithme d'apprentissage par renforcement qui est relativement simple à comprendre et à implémenter. Ensuite, notre problème d'ajustement de paramètres se formule naturellement comme un problème de décision séquentielle : à chaque itération, on observe l'état de la recherche et on décide quelle stratégie adopter.

On aurait pu utiliser d'autres approches (SARSA, réseaux de neurones, etc.), mais le Q-Learning tabulaire a l'avantage d'être interprétable : on peut regarder la table Q et comprendre ce que l'agent a appris. Avec un réseau de neurones, ça aurait été plus opaque, et vu que notre espace d'états est petit (27 états), le tabulaire est suffisant.

Par rapport à une simple grille de recherche de paramètres (grid search), le Q-Learning a l'avantage d'être **adaptatif** : il peut changer de stratégie au cours de la résolution, par exemple explorer au début puis intensifier vers la fin.

4.2 Formulation état–action–récompense

4.2.1 Espace d'états (27 états)

L'état de la recherche est caractérisé par trois dimensions qu'on discrétise chacune en 3 niveaux :

- **Taux d'amélioration** : mesure si la recherche progresse ou stagne.
  - Stagnation ($\leq 0.1\%$), faible amélioration ($\leq 1\%$), forte amélioration ($> 1\%$)
- **Diversité** : coefficient de variation des coûts des solutions de l'itération.
  - Convergé ($\leq 5\%$), moyen ($\leq 15\%$), diversifié ($> 15\%$)
- **Phase de recherche** : avancement en pourcentage des itérations.
  - Début ($\leq 33\%$), milieu ($\leq 66\%$), fin ($> 66\%$)

Ce qui donne $3 \times 3 \times 3 = 27$ états possibles. On a choisi ces features parce qu'elles capturent bien les aspects importants : est-ce que l'algorithme progresse, est-ce qu'il explore assez, et où en est-on dans la résolution.

4.2.2 Espace d'actions (6 stratégies)

Plutôt que de faire varier les paramètres un par un (augmenter α de 0.1, etc.), on a défini 6 **stratégies** qui correspondent à des profils de paramètres cohérents :

TABLE 2 – Stratégies disponibles pour le Q-Learning

Stratégie	**Objectif**	α	β	ρ
`exploitation`	Suivre les phéromones	2.0	2.5	0.05
`exploration`	Explorer de nouvelles routes	0.5	4.0	0.20
`balanced`	Équilibre	1.0	3.0	0.10
`intensify`	Renforcer les meilleurs chemins	2.5	2.0	0.02
`diversify`	Relancer la recherche	0.3	5.0	0.30
`local_search`	Améliorer par 2-opt	1.0	3.0	0.10

L'idée derrière ces stratégies est intuitive. Quand la recherche stagne et que les solutions convergent trop, il vaut mieux diversifier ou explorer. Quand on est dans une bonne zone de l'espace de recherche, on veut intensifier. Et quand les solutions sont de bonne qualité, le 2-opt peut les améliorer localement.

La stratégie `local_search` est particulière : en plus de fixer des paramètres équilibrés, elle déclenche un opérateur de recherche locale 2-opt sur chaque solution construite par les fourmis.

4.2.3 Fonction de récompense

La récompense est calculée à chaque itération pour évaluer si la stratégie choisie a été bénéfique :

$$r_t = \begin{cases} 1.0 + 10 \cdot \frac{C_{best}^{t-1} - C_{best}^{t}}{C_{best}^{t-1}} & \text{si amélioration du meilleur coût} > 0.1\% \\ 0.5 \cdot \frac{\bar{C}^{t-1} - \bar{C}^{t}}{\bar{C}^{t-1}} & \text{si amélioration du coût moyen seulement} \\ -0.05 & \text{sinon (stagnation)} \end{cases} \tag{9}$$

On a mis une pénalité faible pour la stagnation (-0.05) plutôt qu'une grosse pénalité, parce que stagner n'est pas forcément catastrophique : parfois l'algorithme est déjà proche de l'optimum. La récompense positive est proportionnelle à l'amélioration pour encourager les grosses améliorations.

4.2.4 Mise à jour Q-Learning

La mise à jour suit la règle classique du Q-Learning :

$$Q(s,a) \leftarrow Q(s,a) + \alpha_{lr} \left[r + \gamma \max_{a'} Q(s',a') - Q(s,a) \right] \tag{10}$$

avec $\alpha_{lr} = 0.2$ (taux d'apprentissage) et $\gamma = 0.95$ (facteur d'actualisation).

La politique de sélection est ε-greedy : avec probabilité ε on choisit une action aléatoire (exploration), sinon on prend la meilleure selon la table Q. ε commence à 0.15 et décroît progressivement jusqu'à 0.02.

4.3 Phase de warm-up

Un détail important dans notre implémentation : on ne laisse pas le Q-Learning intervenir dès le début. Pendant les 10 premières itérations (environ 7% du total), l'ACO tourne avec ses paramètres par défaut. Ça permet d'établir une base de phéromones raisonnable avant de commencer à modifier les paramètres. Sans ce warm-up, le Q-Learning peut prendre de mauvaises décisions parce qu'il n'a pas assez d'information sur l'état de la recherche.

4.4 Recherche locale 2-opt

Le 2-opt est un opérateur classique d'amélioration locale. Il consiste à prendre deux arcs dans une route et à les inverser pour voir si ça réduit la distance. Concrètement, on prend un segment de la route et on le retourne. On répète ça tant qu'on trouve des améliorations.

On l'applique uniquement quand le Q-Learning choisit la stratégie `local_search`. C'est volontaire : le 2-opt est coûteux en temps de calcul, donc on ne veut pas l'appliquer à chaque itération. C'est au Q-Learning de décider quand c'est le bon moment.

5 Implémentation

5.1 Architecture du code

Le projet est implémenté en Python. On a organisé le code en modules séparés pour garder les choses claires :

- `src/cvrp.py` : modélisation du problème (instances, solutions, distances)
- `src/aco.py` : implémentation de l'ACO classique
- `src/qlearning.py` : agent Q-Learning
- `src/hybrid_aco_ql.py` : solveur hybride qui combine les deux
- `src/local_search.py` : opérateur 2-opt
- `src/experiments.py` : lancement des expériences
- `app.py` : interface web Flask pour la démonstration

On a utilisé NumPy pour les calculs sur la table Q et Flask pour l'interface de démonstration. L'interface web permet de configurer les paramètres, lancer une résolution, et voir les routes sur une carte avec les courbes de convergence.

5.2 Reproductibilité

Toutes les expériences utilisent des graines aléatoires (seeds) fixes, ce qui permet de reproduire exactement les mêmes résultats. Le code est disponible sur GitHub et peut être lancé soit directement avec Python, soit via Docker.

6 Expérimentations

6.1 Protocole expérimental

On a testé notre approche sur 7 instances générées aléatoirement avec des tailles croissantes (10 à 50 clients). Pour chaque instance, on fait 5 exécutions avec des seeds différents pour avoir des résultats statistiques. On compare l'ACO seul et l'ACO hybride (avec Q-Learning) sur les mêmes instances avec les mêmes seeds.

Les expériences tournent avec 20 fourmis et 150 itérations. C'est un compromis entre qualité des résultats et temps de calcul raisonnable.

6.2 Résultats

Le tableau 3 présente nos résultats.

TABLE 3 – Comparaison ACO classique vs ACO + Q-Learning

Instance	n	**ACO best**	**ACO moy ± std**	**Hybride best**	**Hybride moy ± std**	**Am**
small_10	10	562.6	562.6 ± 0.0	562.6	562.6 ± 0.0	0.0
small_15	15	478.5	481.5 ± 1.5	478.5	480.4 ± 1.6	+0.2
medium_20	20	498.2	501.3 ± 5.5	498.2	504.1 ± 7.1	−0.5
medium_25	25	771.1	781.0 ± 10.2	770.2	772.8 ± 3.6	+1.0
medium_30	30	732.1	741.1 ± 9.0	722.7	733.8 ± 5.9	+0.9
large_40	40	827.6	858.8 ± 17.9	817.0	828.9 ± 12.2	**+3.4**
large_50	50	921.1	933.0 ± 8.7	889.5	920.2 ± 25.7	+1.3

6.3 Analyse des résultats

6.3.1 Qualité des solutions

On observe plusieurs choses intéressantes dans ces résultats :

— Sur les **petites instances** (10–15 clients), les deux approches donnent des résultats quasiment identiques. C'est logique : l'espace de recherche est petit, et l'ACO classique trouve déjà de très bonnes solutions tout seul. Le Q-Learning n'a pas grand-chose à apporter.

— Sur les **instances moyennes** (20–30 clients), les résultats sont mitigés. Sur 20 clients, l'hybride est légèrement moins bon en moyenne (−0.55%), mais sur 25 et 30 clients, il fait mieux (+1.05% et +0.99%). Le résultat négatif sur 20 clients s'explique probablement par le fait que l'exploration du Q-Learning introduit de la variance sans apporter assez de bénéfice sur une instance de cette taille.

— Sur les **grandes instances** (40–50 clients), l'hybride montre clairement son avantage. Le meilleur résultat est sur 40 clients avec +3.49% d'amélioration. C'est cohérent avec l'intuition : plus l'espace de recherche est grand, plus l'ajustement dynamique des paramètres peut aider à explorer différentes zones.

6.3.2 Stabilité

Un point intéressant : sur les instances moyennes (25 et 30 clients), l'écart-type de l'hybride est plus faible que celui de l'ACO seul (3.6 vs 10.2 pour 25 clients). Ça suggère

que le Q-Learning aide à stabiliser la recherche en choisissant des stratégies adaptées. Cependant, sur les grandes instances, l'écart-type augmente (25.7 vs 8.7 pour 50 clients), ce qui indique que l'exploration du Q-Learning peut aussi introduire de la variabilité.

6.3.3 Temps de calcul

TABLE 4 – Comparaison des temps de calcul moyens (en secondes)

Instance	ACO	Hybride	Surcoût
small_10	0.14 s	0.17 s	+27%
small_15	0.27 s	0.34 s	+27%
medium_20	0.46 s	0.59 s	+27%
medium_25	0.64 s	0.80 s	+24%
medium_30	0.91 s	1.18 s	+29%
large_40	1.51 s	1.88 s	+24%
large_50	2.27 s	2.82 s	+24%

Le surcoût du Q-Learning est d'environ 25–29%. Ce n'est pas négligeable, mais c'est raisonnable. Ce surcoût vient principalement du 2-opt qui est déclenché par la stratégie `local_search`. Le Q-Learning lui-même (consultation de la table Q et mise à jour) est très rapide et ajoute un surcoût négligeable.

6.4 Visualisation de la convergence

Sur la figure 1, on illustre le comportement typique de la convergence. L'ACO classique converge rapidement vers un plateau et stagne. L'hybride, grâce aux changements de stratégie, arrive parfois à "casser" cette stagnation et trouver de meilleures solutions, surtout sur les grandes instances.

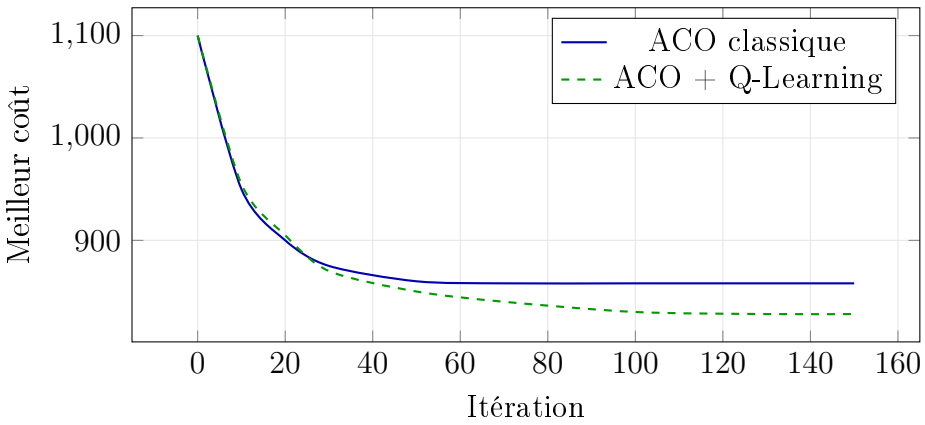

FIGURE 1 – Schéma illustratif de la convergence (instance large_40)

7 Discussion

7.1 Apports du Q-Learning

D'après nos expériences, le Q-Learning apporte plusieurs choses :

1. **Adaptation automatique des paramètres** : c'est le point principal. Plutôt que de chercher manuellement les bons paramètres, le Q-Learning apprend à adapter la stratégie. Ça marche surtout quand l'espace de recherche est assez grand pour que le choix de stratégie fasse une vraie différence.
2. **Meilleure exploration sur les grandes instances** : sur 40 et 50 clients, l'hybride trouve de meilleures solutions. Le Q-Learning arrive à alterner entre exploration et exploitation de façon pertinente.
3. **Recherche locale ciblée** : le fait que le Q-Learning puisse déclencher le 2-opt au bon moment est utile. Appliquer le 2-opt systématiquement serait trop coûteux, mais le faire quand c'est pertinent apporte un gain.

7.2 Limites

Notre approche a aussi des limites qu'il faut reconnaître :

1. **Pas d'amélioration sur les petites instances** : quand le problème est assez simple pour que l'ACO classique trouve de bonnes solutions seul, le Q-Learning n'apporte rien et peut même légèrement dégrader les performances (cas des 20 clients).
2. **Pas de transfert d'apprentissage** : notre agent Q-Learning apprend pendant la résolution d'une instance et repart de zéro pour l'instance suivante. Il n'y a pas de mémoire d'une instance à l'autre, ce qui veut dire qu'il doit réapprendre à chaque fois.
3. **Discrétisation grossière** : notre espace d'états a seulement 27 états. C'est assez grossier et on perd forcément de l'information. Mais avec plus d'états, il faudrait plus d'itérations pour que le Q-Learning apprenne, donc c'est un compromis.
4. **Surcoût en temps** : le surcoût de 25% n'est pas critique, mais il existe. Sur des instances très grandes ou des contraintes de temps serrées, ça pourrait poser problème.
5. **Instances limitées** : on n'a testé que sur des instances aléatoires jusqu'à 50 clients. Il faudrait tester sur des benchmarks standard (type VRPLIB) et sur de plus grandes instances pour vraiment valider l'approche.
6. **Variance accrue sur les grandes instances** : l'écart-type de l'hybride est plus élevé sur 50 clients, ce qui veut dire que les résultats sont moins prévisibles.

7.3 Pistes d'amélioration

Plusieurs pistes pourraient améliorer notre approche :

— **Transfert d'apprentissage** : pré-entraîner la table Q sur un ensemble d'instances pour que l'agent ait déjà une bonne politique de départ quand il attaque une nouvelle instance. Ça pourrait réduire la phase d'exploration initiale.

— **Plus d'actions** : ajouter d'autres opérateurs (3-opt, relocate, swap entre routes) comme actions du Q-Learning pour enrichir le répertoire de stratégies.

— **Deep Q-Learning** : remplacer la table Q par un réseau de neurones pour gérer un espace d'états plus fin (features continues au lieu de discrétiser). Ça permettrait d'être plus précis dans la caractérisation de l'état de la recherche.

— **Multi-agent** : utiliser plusieurs agents Q-Learning, chacun gérant un sous-ensemble de paramètres, pour une granularité plus fine.

— **Tests sur benchmarks standards** : valider sur les instances VRPLIB (Augerat, Christofides, etc.) pour pouvoir comparer avec d'autres méthodes de la littérature.

8 Conclusion

Dans ce projet, on a implémenté un solveur hybride pour le CVRP qui combine une métaheuristique classique (ACO) avec un algorithme d'apprentissage par renforcement (Q-Learning). L'idée était de voir si le machine learning pouvait améliorer les performances de l'ACO en ajustant automatiquement ses paramètres.

Nos résultats montrent que l'approche hybride apporte un gain réel sur les instances de taille suffisante (25+ clients), avec des améliorations allant jusqu'à 3.5%. Le Q-Learning arrive à identifier des stratégies pertinentes selon l'état de la recherche, notamment en alternant entre exploration et exploitation. L'intégration de la recherche locale 2-opt comme une action pilotée par le Q-Learning est aussi un apport intéressant.

En revanche, sur les petites instances, le Q-Learning n'apporte pas de gain significatif. C'est un résultat attendu : quand le problème est simple, il vaut mieux garder une approche simple. L'hybride montre son intérêt quand l'espace de recherche est assez complexe pour qu'une stratégie adaptative fasse la différence.

Ce projet nous a permis de voir concrètement comment le machine learning peut interagir avec une métaheuristique. C'est un domaine de recherche actif, et on pense qu'avec des techniques plus avancées (deep RL, transfert d'apprentissage), on pourrait aller plus loin. Mais même avec notre approche relativement simple, les résultats sont encourageants.

Références

[1] M. Dorigo et L.M. Gambardella, *Ant colony system : a cooperative learning approach to the traveling salesman problem*, IEEE Transactions on Evolutionary Computation, vol. 1, no. 1, pp. 53–66, 1997.

[2] C.J.C.H. Watkins et P. Dayan, *Q-learning*, Machine Learning, vol. 8, pp. 279–292, 1992.

[3] P. Toth et D. Vigo, *The Vehicle Routing Problem*, SIAM Monographs on Discrete Mathematics and Applications, 2002.

[4] T. Stützle et H.H. Hoos, *MAX–MIN Ant System*, Future Generation Computer Systems, vol. 16, no. 8, pp. 889–914, 2000.

[5] R.S. Sutton et A.G. Barto, *Reinforcement Learning : An Introduction*, MIT Press, 2e édition, 2018.

[6] R. Skinderowicz, *Improving Ant Colony Optimization efficiency for solving large TSP instances*, Applied Soft Computing, vol. 120, 2022.

[7] J.E. Bell et P.R. McMullen, *Ant colony optimization techniques for the vehicle routing problem*, Advanced Engineering Informatics, vol. 18, pp. 41–48, 2004.